

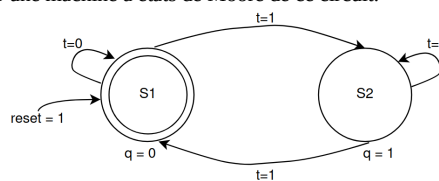
SN360/SN361 : Introduction à la conception des circuits numériques

Exercice 1 :

Vous allez décrire en VHDL une bascule Toggle dont la sortie q change de valeur à chaque nouveau front d'horloge clk quand t est actif comme le montre la table ci-dessous. Le signal de remise à zéro $reset$ est asynchrone.

reset	clk	t	q^*
1	-	-	0
0	0	-	q
0	1	-	q
0	$\downarrow$	0	q
0	$\downarrow$	1	q'

- 1) Donner le diagramme de transitions d'une machine à états de Moore de ce circuit.



- 2) Donner le code VHDL de cette machine (décrire uniquement son architecture) avec 2 processus comme vu dans le cours.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Toggle_FlipFlop_Moore is
  port (
    clk : in std_logic;
    reset : in std_logic;
    t : in std_logic;
    q : out std_logic
  );
end Toggle_FlipFlop_Moore;

architecture Behavioral of Toggle_FlipFlop_Moore is
  type state_type is (S0, S1);
  signal current_state, next_state : state_type := S0;
begin

```

```

  process(clk, reset)
  begin
    if reset = '1' then
      current_state <= S0;
    elsif rising_edge(clk) then
      current_state <= next_state;
    end if;
  end process;

```

```

  process(current_state, t)
  begin
    next_state <= current_state;
    case current_state is
      when S0 =>
        q <= '0';
        if t = '1' then
          next_state <= S1;
        end if;
      when S1 =>
        q <= '1';
        if t = '1' then
          next_state <= S0;
        end if;
      when others =>

```

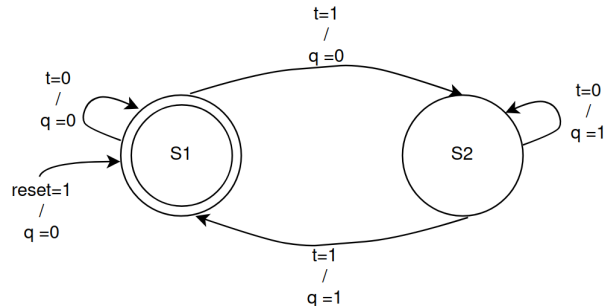
SN360/SN361 : Introduction à la conception des circuits numériques

```

    next_state <= S0;
  end case;
end process;
end Behavioral;

```

3) Faire de même (questions 1 et 2) avec une machine de Mealy.



```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity Toggle_FlipFlop_Mealy is

```

```

  port (
    clk : in std_logic;
    reset : in std_logic;
    t : in std_logic;
    q : out std_logic
  );

```

```

end Toggle_FlipFlop_Mealy;

```

```

architecture Behavioral of Toggle_FlipFlop_Mealy is

```

```

  type state_type is (S0, S1);
  signal current_state, next_state : state_type := S0;

```

```

begin

```

```

  process(clk, reset)
  begin
    if reset = '1' then
      current_state <= S0;
    elsif rising_edge(clk) then
      current_state <= next_state;
    end if;
  end process;

```

```

end process;

```

```

process(current_state, t)

```

```

begin

```

```

  case current_state is
    when S0 =>
      if t = '1' then
        next_state <= S1;
        q <= '0';
      else
        next_state <= S0;
        q <= '0';
      end if;
    when S1 =>
      if t = '1' then
        next_state <= S0;
        q <= '1';
      else

```

```
        next_state <= S1; -- Stay in S1 if t = 0
        q <= '1';
    end if;
    when others =>
        next_state <= S0;
        q <= '0';
    end case;
end process;
end Behavioral;
```

4) Combien de bascules nécessite l'implantation de chaque machine ?

Une bascule pour les deux.

On suppose maintenant que le signal t est asynchrone.

1. Afin d'éviter l'apparition de métastabilité, décrire en VHDL un unique processus permettant de synchroniser le signal asynchrone t.